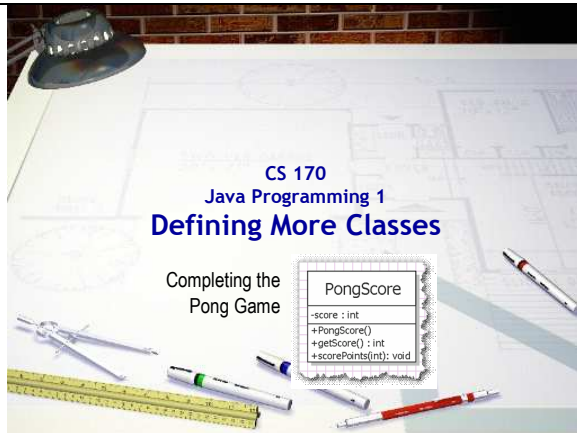




Slide 1

CS 170

Java Programming 1

Defining More Classes

Duration: 00:00:43

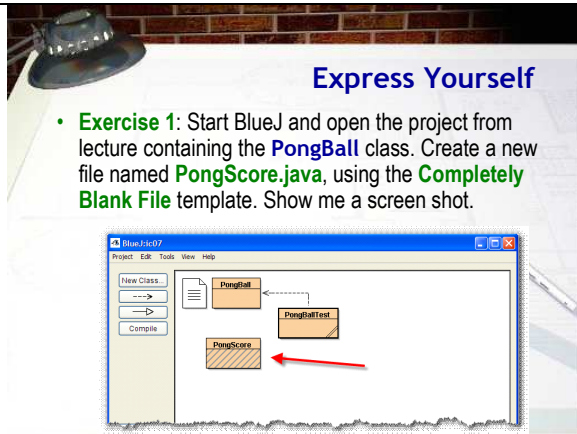
Advance mode: Auto

Notes:

Hi there. Welcome to the CS 170, Java Programming 1 lecture, Defining More Classes.

In class this week we walked through the steps needed to define your first class, the PongBall class. In this lesson, we'll pick up where you left off with the PongScore class which keeps track of each player's score as the game progresses.

This online lecture will be a little different than the in-class version, though. Since we'll be reviewing the material covered in class, I'll go through the steps a little faster, and ask you to do more of the work. If you can't remember exactly how to do one portion, refer back to the lecture slides from class this week.



Slide 2

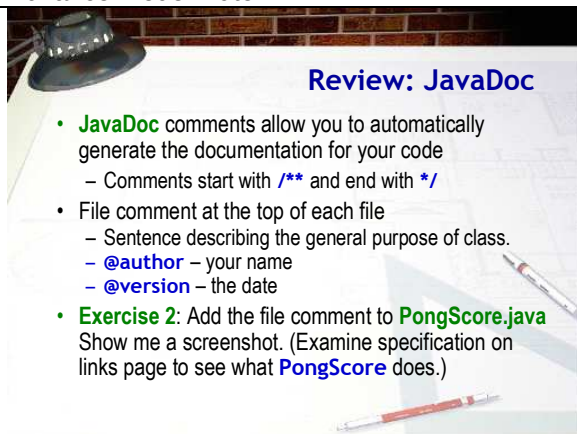
Express Yourself

Duration: 00:00:32

Advance mode: Auto

Notes:

Open your in-class lab exercise document and start a new section for this mini-lecture. For the first exercise, open the project you created at school that contains your PongBall and PongBallTest classes. Then, create a next file named PongScore.java using the Completely Blank File template. Show me a screenshot of the BlueJ project window.



Slide 3

Review: JavaDoc

Duration: 00:01:08

Notes:

Let's start by reviewing the JavaDoc comments. Remember that JavaDoc comments allow you to automatically generate user documentation using only the comments from your code. JavaDoc comments always start with a forward slash, followed by two asterisks. You can't have any spaces between any of these three characters. The JavaDoc comment ends with a */ just like a regular multi-line Java comment.

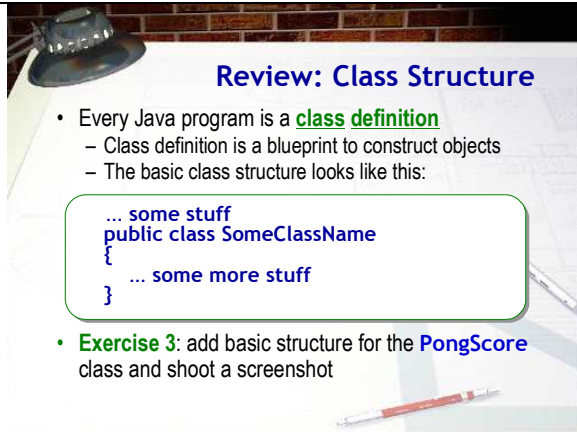
You'll use JavaDoc comments for every class you create as well as all of the methods in the class. At the top of every file you should add a JavaDoc comment that includes:

- A sentence describing the general purpose of the class.
- An @author tag that is followed by your name

Advance mode: Auto

- An @version tag that includes the date the file was updated

For Exercise 2, add the file comment to PongScore.java and show me a screenshot. To find out exactly what the purpose of the class is, check the program specification on the links page for this week.



Review: Class Structure

- Every Java program is a **class definition**
 - Class definition is a blueprint to construct objects
 - The basic class structure looks like this:

```
... some stuff
public class SomeClassName
{
    ... some more stuff
}
```

- **Exercise 3:** add basic structure for the **PongScore** class and shoot a screenshot

Slide 4

Review: Class Structure

Duration: 00:01:20

Advance mode: Auto

Notes:

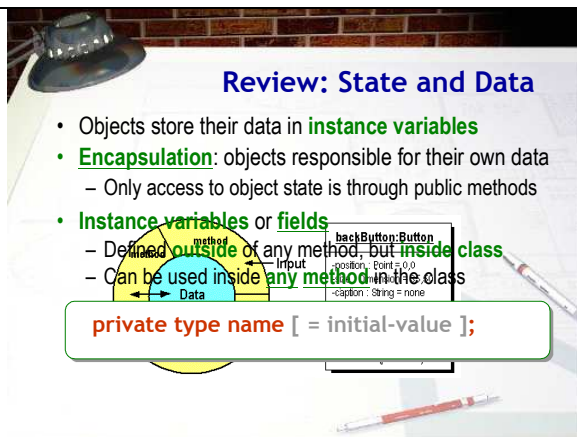
Alright, now let's turn to the code itself.

You'll recall that every Java program is a class definition, and that that class definition is used when you want to construct objects by using the new operator. The class definition will contain the attributes and methods that make up the class. In our case, the class definition will be used when the Pong game wants to construct a PongScore object.

The basic structure for a class looks like this:

- there's some optional stuff before the class starts. This includes comments and import statements. You won't need any import statements so you'll only have comments before the class.
- the class itself consists of a header and a body. The header consists of the words public class followed by the name of the class (which much match the name of the file that contains the definition). The body consists of a pair of curly braces which contain or delimit the attributes and methods of the class: more stuff in other words.

For Exercise 3, add the basic structure for the PongScore class, make sure you can compile, and then shoot a screenshot.



Review: State and Data

- Objects store their data in **instance variables**
- **Encapsulation:** objects responsible for their own data
 - Only access to object state is through public methods
- **Instance variables or fields**
 - Defined **outside** of any method, but **inside class**
 - Can be used inside **any method** in the class

```
private type name [ = initial-value ];
```

Slide 5

Review: State and Data

Duration: 00:02:28

Advance mode: Auto

Notes:

Now that we have the structure of your class completed, let's look at what goes inside. You remember from earlier lectures, that a class contains the definitions for two things:

- the data attributes that make up each object state
- the object's behaviors defined by its methods

Objects store their state in data members called instance variables or fields, which are "hidden" inside the object.

This "data hiding", called encapsulation, is one of three fundamental principles used with well-designed object-oriented programs. When you design an object-oriented program, you want each object to be responsible for its own data. Instead of letting other parts of the program directly access the data fields in your objects, you design accessor and mutator methods that allow you complete control over

how the object's state is changed and used.

Here's a UML object diagram that illustrates encapsulation. Notice that the Button object, backButton has fields to store its position and size. These fields aren't directly accessible, though. To change the position of the backButton object, you'll need to call the setPosition() method.

Instance variables are defined outside of all methods, but inside the class. A common mistake that students often make is to try to define an instance variable after the closing brace that ends the class, which is illegal. It doesn't make any difference if your instance variables are defined at the beginning of the class or the end of the class. When you define a local variable, it can only be used inside the method you defined it in; when you define an instance variable, it can be used from within any method in your class.

Here's the syntax for defining an instance variable. The syntax is almost exactly the same as that used for defining a local variable except for the use of the access specifier private. We'll always make our instance variables private, although it's legal to make them public or protected. We'll cover protected access later when we talk about inheritance.

With a local variable, if you leave off the optional initializer (in gray), you can't use the variable before you assign it a value. With instance variables, though, if you leave off the initializer, then the variable is automatically initialized to 0.

- One instance variable representing the current score for each player
- **Exercise 4:** add the definition for the **PongScore** instance variable, and shoot a screenshot.
 - Add a JavaDoc comment before the variable explaining what it represents.

```
classDiagram
    class PongScore {
        -score : int
        +PongScore()
        +getScore() : int
        +scorePoints(int): void
    }
```

Slide 6
PongScore Fields
Duration: 00:00:34
Advance mode: Auto

Notes:

So let's put that to work. Here's the UML diagram for the PongScore class. As you can see, there is one instance variable that represents the score for each individual player. For Exercise 4, add the definition for the PongScore instance variable and shoot me a screen-shot of your code. Make sure you add a JavaDoc comment right before the variable that explains what it represents.

Review: Methods

- Methods that **modify** an object are called **mutators**
- Accessors** **return information about** an object

```
public void methodName( parameters )
{
    // statements
}
```

Mutator

```
public returnType methodName( parameters )
{
    // statements
    return typeExpression;
}
```

Accessor

Slide 7

Review: Methods

Duration: 00:01:22

Advance mode: Auto

Notes:

After the instance variables, the other thing that every class contains are its methods. Methods define the actions or behavior that each object can perform. Methods can carry out a calculation, perform an action, modify an object, or provide information about an object.

The methods that modify an object are called mutators. If you want to change the location of a Rectangle, you use the setLocation() mutator method. Mutators have this general form:

- the return type is void
- mutators may have parameters, but they don't have to
- you don't need a return statement.

Accessors are methods that return information about an object. If you want to find out how many characters are in a String, you use the length() accessor method. Accessors have this general form:

- the return type is anything other than void
- accessors may have parameters, but they don't have to
- accessors always contain a return statement. In a return statement, the expression following the keyword return must be the same "kind of thing" as the return type specified in the method header.

Let's look at the PongScore accessor methods.

Review: Writing Accessors

- Exercise 5:** find the accessor in the **PongScore** class and write the method along with a **Javadoc** comment.
 - Your Javadoc comment should have a sentence explaining what the method does as well as use the **@return** tag to explain what the returned value represents.

```
class PongScore
{
    -score : int
    +PongScore()
    +getScore() : int
    +scorePoints(int): void
}
```

Slide 8

Review: Writing Accessors

Duration: 00:00:41

Advance mode: Auto

Notes:

Now it's your turn again. Using what you've learned, find the accessor in the PongScore class, and, for Exercise 5, write the method along with a Javadoc comment. Remember that when you write the Javadoc comment for an accessor, you'll need to include the @return tag to explain what the returned value represents. If you need help on figuring out what the accessor should do, remember that the Pong game specification (available from the class Web page) will have more information for you.

Shoot me a screen-shot showing the code once you're finished.

Review: Parameters

- Extra information passed **into** your method when called
 - Formal parameters** are variables defined in heading
 - Actual parameters** are the values supplied calling
- Formal parameter is declared **inside** the method header
 - You include a name and type for **each** argument

```
public void myMethod(int par1, int par2)
{
}
```

Type Name

Slide 9

Review: Parameters

Duration: 00:01:02

Advance mode: Auto

Notes:

Many of the methods you write need some extra information to carry out their task. That extra information is passed to the method when you call it by using parameters. Formal parameters are variables that you'll define along with the method heading. Formal parameters hold the values supplied when calling a method. Those values are called actual parameters.

A formal parameter is just another kind of local variable. Instead of defining the variable inside the method body, you define it inside the parentheses following the method name, an area known as the parameter list. For each parameter variable you define you have to specify both its type and give it a name. Here's an example.

The method myMethod has two formal parameter variables. The type of both is int, and their names are par1 and par2.

PongScore Mutator

- Pong program supplies actual parameters, you need to supply **formal** parameters for the **PongScore** mutator method.
- Exercise 6:** find the mutator in the **PongScore** class and write the method along with a **JavaDoc** comment using the **@param** tag for each parameter.

```
PongScore
-score : int
+PongScore()
+getScore() : int
+scorePoints(int): void
```

Slide 10

PongScore Mutator

Duration: 00:00:46

Advance mode: Auto

Notes:

The PongScore class has one mutator method that changes the PongScore object's field. When the Pong program creates a PongScore object, it calls the mutator method and supplies the actual parameter values. It's up to you, though, to define the formal parameter variable when you write the mutator method.

For Exercise 6, find the mutator method in the PongScore class and write its JavaDoc comment. Since this mutator has one parameter, your documentation will need to include an @param tag describing what the parameter does. (Remember to check the specification for the PongScore class.)

Review: Constructor

- Instance variables are **automatically initialized**
 - Given values **0**, **false**, or **null** depending on type
- Normally want to **explicitly** initialize your objects
- This is done by writing a **constructor**
- Constructor look like methods, with two constraints:
 - The name is **always** the same as the class
 - The constructor **cannot** have a return type
- Exercise 7:** write and doc **PongScore** constructor

Slide 11

Review: Constructor

Duration: 00:01:23

Advance mode: Auto

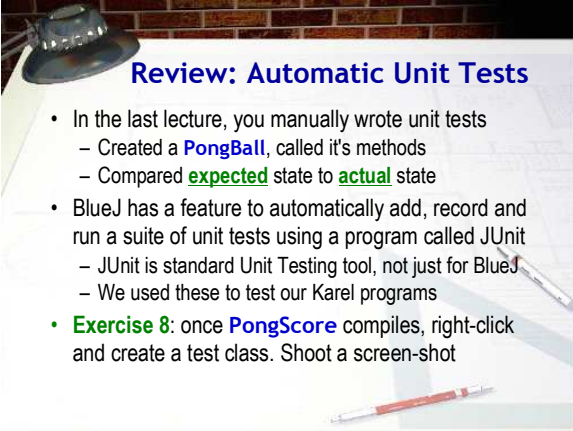
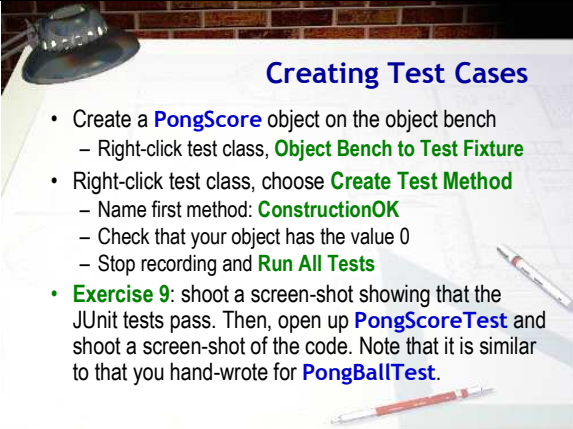
Notes:

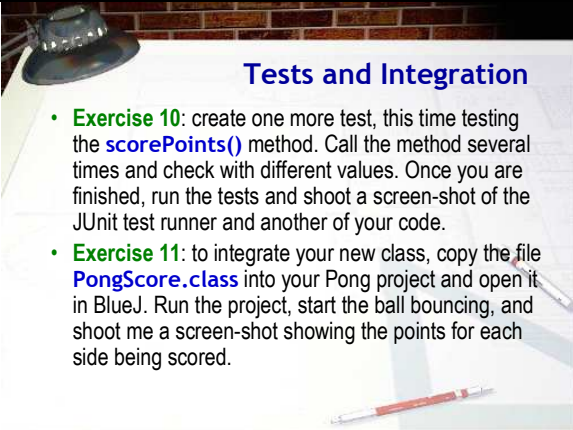
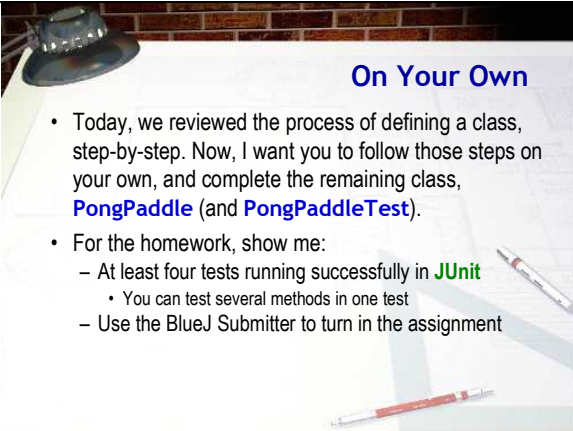
The last thing we need to take care of in your class is the constructor. As I mentioned earlier, instance variables are automatically initialized (if you don't supply an initial value), unlike local variables. Even if you do supply an initial value when defining your instance variables, though, usually you'll want to explicitly initialize your objects. This is done by writing a constructor.

A constructor looks just like a method except that the constructor:

- always has the same name as its class, and
- a constructor never ever has a return type

By the way, students often try to use void as the constructor return type, and are surprised when the Java compiler says that's OK. If you use a return type, though, you are writing a

	method, not a constructor, and that method won't be used to initialize your object like a constructor would. Every time you construct a PongScore object, its score field should be initialized to zero. Since that's the default behavior for uninitialized instance variables, we really don't need a constructor. Just to give you some practice writing one, though, for Exercise 7, write an document the PongScore constructor, explicitly initializing the field inside the constructor.
 Slide 12 🎧 Review: Automatic Unit Tests Duration: 00:01:07 Advance mode: Auto	Notes: During the in-class lecture this week, you learned how to manually write a test program. There you: • created a PongBall object • called its mutator methods, like move() and bounceX() • called its accessor methods, like getX() and getY() • compared the actual values you got back with the values you expected based on the class specification. As you've already seen in the last online lecture, though, BlueJ will automatically create and run a set of unit tests using the built-in testing tool called JUnit. JUnit is not really a part of BlueJ; it's the standard unit testing tool used almost everywhere in the Java world. You got a little experience using JUnit to test your Karel program in the last lecture. For Exercise 8 (once PongScore compiles), right-click PongScore and choose "Create a Test Class". Shoot me a screenshot showing both PongScore and PongScoreTest.
 Slide 13 🎧 Creating Test Cases Duration: 00:01:23 Advance mode: Auto	Notes: To write your unit tests for PongScore, first you'll need to create the test fixture. To do that, just create a PongScore object on the object bench. Then, right-click the test class, and select Object Bench to Test Fixture. Remember that you can double-check your work by looking at the setUp() method. Now, let's write the first test method. * Right-click your test class and choose Create Test method * Name the method ConstructionOK After the object bench is populated (by automatically running the setUp() method), right-click your object and check to make sure that it has the value 0, using an appropriate assertion. Then, immediately press End Recording to stop the test recording. To run the tests, just click the Run All Tests button. For Exercise 9, shoot me a screen-shot showing that your single JUnit tests passes. Then, open up PongScoreTest and

	shoot a screen-shot of the code. You should notice how similar it is, conceptually, to the code you wrote for PongBallTest. Just like PongBallTest, the JUnit testing code creates an object and then calls some methods to change and inspect the object's state.
 Slide 14 🎧 Tests and Integration Duration: 00:01:21 Advance mode: Auto	Notes: Two more exercises and we'll be done. For Exercise 10, create one more test, this time testing the <code>scorePoints()</code> method. You'll want to call the method several times, with different values each time, and then check to make sure that the accumulated score contains the correct value. Once you're finished, run the tests and then shoot a screen-shot of the JUnit GUI Test Runner and a second picture of your code. To put the PongScore class to work, you need to integrate it into the Pong project (that's the project with only .class files that we worked with in class). Copy the PongScore.class file from your project and drop it into the folder containing the Pong classes. Your new class should be larger than the original PongScore.class file that it replaces. Run the project by right-clicking the Pong class in the BlueJ IDE and selecting "Run Main Method" from the bottom of the menu. Press the B key to get the ball rolling. Let it bounce against the wall a few times, and shoot me a screen-shot showing the individual PongScore objects at the bottom of the screen racking up the points.
 Slide 15 🎧 On Your Own Duration: 00:00:47 Advance mode: Auto	Notes: In class this week, you learned how to define a class and we all went, step-by-step through the process of creating and testing the PongBall class. In this lecture, we reviewed each of the steps required to define a class, and you worked a little more independently, writing the accessors, constructors and mutators by yourself. Now it's time to go on your own. Follow the same steps you used for PongBall and then PongScore to complete the PongPaddle homework assignment this week. You'll use the BlueJ Submitter program to turn in the PongPaddle and PongPaddleTest program to Web-Cat. You'll find more information on this week's Exercises page.